

Warden

A migration guard that doesn't just roll back — it tells you what the migration was *trying* to do.

v153

Before — table healthy

× **CHECK**

Constraint violated by 1 row

v153

Healed — time-travel restore

Every figure in this deck is real output from a live Databricks warehouse, captured today.

A failed migration reverts the data and forgets the reason.

Migrations fail in production constantly. The standard response is a blind revert: the schema goes back, and every trace of *why* the change was attempted disappears with it.

The engineer picking up the incident reconstructs intent from a diff and a commit message — at the worst possible moment, under time pressure.

Git shows what changed. It doesn't show what you were trying to achieve.

Warden's user: anyone shipping schema or data migrations who wants recovery that explains itself.

Checkpoint → impact → apply → validate → heal → explain

01 · READ INTENT

Entire checkpoint

Pull the recorded intent for this change via `entire checkpoint explain` — before anything touches the warehouse.

02 · BLAST RADIUS

Entire Graph impact

Run impact analysis on the affected symbol first, so the risk is known before it's taken.

03 · APPLY

Delta migration

Record the pre-migration Delta version, then execute.

04 · VALIDATE

Fail fast

Delta's own `CHECK` enforcement catches the bad row — validation is not our code guessing.

05 · HEAL

Time-travel restore

`RESTORE TABLE ... TO VERSION AS OF` — a metadata operation, not a data rewrite.

06 · EXPLAIN

Intent, locally

Surface the recorded intent to the operator so the next migration is corrected, not re-attempted blindly.

The whole loop, against a real warehouse

```
[warden] 'warden.demo_users' currently at Delta version 153 [warden] FAILURE:
[DELTA_NEW_CHECK_CONSTRAINT_VIOLATION] 1 rows in workspace.warden.demo_users violate the new
CHECK constraint (plan IN ('free', 'pro', 'enterprise')) [warden] healing: restoring
'warden.demo_users' to Delta version 153 via time-travel... — most recent row in
warden.migration_log — status='healed' context_completeness='complete' note='migration
failed (ServerOperationError); rolled back warden.demo_users to Delta version 153. Checkpoint
intent and full error detail kept local-only (context: complete); see operator console for
detail.' — post-migration Delta version — Delta version: 153 ← matches pre-migration: the
heal actually happened
```

Note what is *absent* from that log row: no raw checkpoint intent, and no raw database error text — only the exception's type name. That is the Curveball response, visible in production data.

Checkpoint-*driven*, not checkpoint-*logged*.

The distinction is the whole product. Plenty of tools write a checkpoint as a side effect. Warden **reads** one to make the recovery useful.

- **Without the checkpoint** — a revert, and an engineer reconstructing intent by hand.
- **With it** — a recovery that states what was attempted and why, so the corrected migration is the next step.

The healing decision depends on the checkpoint. Remove Entire and the feature is gone, not degraded.

The assumption that broke

We had been writing the full checkpoint intent into `migration_log` — a Databricks table, outside Entire's local boundary. "More context in the log is better" became wrong.

FIX 01

context_completeness

Every row is labelled `complete` / `redacted` / `unavailable`, so partial context can never be mistaken for whole.

FIX 02

LocalOnlyText

Intent is wrapped so leaking it is a *type* error. Its `str()` returns a placeholder — a careless future call site gets a placeholder, not a leak.

FIX 03

error_summary()

Reviewing our own fix found the same leak through another door: Delta errors can embed row values. Exceptions now cross the boundary as a type name only.

Fix 03 wasn't in the brief. It came from asking where *else* the same shape of leak could occur — and it was already there.

The Graph named the exact function the Curveball was about

```
$ entire graph diff --base b8c9fe619 --head HEAD warden/migrate.py (Python) + class LocalOnlyText added + function classify_completeness added + function error_summary added ~ function get_latest_checkpoint body changed (6 dependents) ~ function log_attempt signature changed (2 dependents) ~ function apply_migration body changed (4 dependents)
```

- **That signature change *is* the privacy boundary** — `log_attempt` gained the `context_completeness` parameter, and it is the one function that writes to the external service.
- **Its 2 dependents** are exactly the applied and healed call sites we had to change — each covered by a test.
- **Verified, not quoted** — every edge named here is pinned by a test in `test_migrate.py`.

A fresh session really did resume from a checkpoint — this one.

The rubric asks whether a fresh session can pick the work up from a checkpoint. We didn't stage a demonstration of that. It is *how the Curveball response was built*, and the commit history shows it:

0d29a02

Prior session deliberately stopped at a stable point

cold start

New session, no memory — first action: checkpoint explain

cc16573 →

Reconstructed intent, ran impact, shipped the privacy boundary

It reconstructed the architecture, located the offending assumption, ran Graph impact *before* editing, and implemented the fix with tests. Every commit from **cc16573cc** onward is that session's output. **warden/resume_report.py** is that capability productised — it tells an operator whether context is complete enough to proceed, without exposing the checkpoint text itself.

What is load-bearing, and what isn't

GENUINELY DATABRICKS

- **Delta time-travel is the rollback.** `RESTORE TABLE ... TO VERSION AS OF` — metadata-only, which is why it scales to large tables.
- **Delta CHECK enforcement is the failure trigger.** The constraint validates existing rows at `ADD CONSTRAINT` time — the migration fails there, not in our code.
- **Lakeview dashboard** — a real workspace object over `migration_log`, built through the REST API.

PORTABLE, AND WE'LL SAY SO

- **The connector and logging path** would work against any SQL warehouse. We're not claiming otherwise.
- **Synthetic data only** — 50 generated users on the reserved `example.invalid` domain; user 007 carries `plan='legacy'` so the failure is deterministic.

Four situations this already fits

HIGHEST LEVERAGE

Agent-authored migrations

An AI agent proposes a schema change. Warden is the guard rail: the agent's own recorded intent becomes the explanation when the migration fails — and the privacy boundary keeps that intent off the warehouse.

DATA QUALITY

Rolling a rule onto legacy data

Exactly the demo: a business rule meets rows that predate it. You want the offending rows named and the table intact, not a half-applied constraint.

INCIDENT RESPONSE

3am schema rollback

The engineer who picks up the page is not the one who wrote the migration. The recorded intent is the difference between a five-minute fix and an hour of archaeology.

COMPLIANCE

Auditable change history

`migration_log` records every attempt, its outcome, and how complete the context was — deliberately without the sensitive text itself.

What we did not build

- **One table, one migration, one validation path.** A narrow vertical slice proven end-to-end, chosen over a broad one that doesn't run.
- **A multi-table scheduler exists — and is excluded.** We prototyped it today and left it out because it wasn't live-verified before the deadline. It's on the `handoff` branch, and the docs say exactly why it isn't shipped.
- **RESTORE fails if VACUUM purged the target version.** Production needs a retention policy that guarantees the rollback window survives.
- **Validation is a full-table scan**, and the token is a long-lived PAT. Both need to change before this is a production tool.

19

tests passing

153

Delta version, pre = post

2

independent review passes

1

privacy test proven by mutation

From one table to a migration control plane

- **Multi-table, dependency-aware parallelism** — independent tables migrating concurrently under a worker bound; same-table work strictly ordered; a failure isolated to its own table.
- **Live progress instead of a terminal pass/fail** — rows processed, replay lag, phase, warehouse spend. That's also what makes health alerts meaningful mid-flight.
- **An operator in the loop** — inspect what validation caught, correct the offending rows, approve or hold a cutover. The checkpoint boundary is what makes exposing that safe.
- **The migrations that actually frighten people** — changing a primary key, splitting one table into several, merging several into one. A blind revert is least acceptable exactly where it's most likely needed.

Verify any of this yourself: `bash warden/demo.sh` for the live loop, or `bash warden/demo.sh --self-test` with no credentials at all.